

The very basic & informal story about AGM

Discrete logarithm - what we base cryptography on

Standard Model Proofs - Non restrictive model, but not always able to prove security in this model

GGM proofs - very restrictive model, but enables some proofs.

AGM proofs - less restrictive than GGM more restrictive than Standard Model.

Structure of this presentation

- Recap Discrete log + Standard model
- GGM
- AGM
 - o Algebraic algorithms
 - o AGM compared to GGM
 - o The model
- Example security of BLS

Discrete logarithm

~informal

Given a cyclic group G of prime order p and generator g .

Then given $h = g^x$, it is hard to find x .

From Dlog many other cryptographic assumptions follow

- DDH
- CDH
- ...

Adversary is only limited in time & computational power

Example of proof in the standard model

$e \xrightarrow{g, g^x} \beta$

simulates to protocol to prove to A

Reduce forgery to answer to dlog

breaking protocol

idea of AGM transform

$A \rightarrow A_{aug}$

gets more than just a forgery thus β has more tools in its reduction

Generic Group Model

[Sho 97], [Mau 05]

I liked this definition

Intuition

Generic group algorithms are algorithms that don't exploit any special structure of the group and only uses abstract group operations.

Group is modeled as a black box

The Generic Group Model

2.1 The Model

We consider an abstract model of computation characterized by a black-box $\mathbf{B}$ which can store values from a certain set S (e.g. a group) in internal state variables $V_1, V_2, \dots, V_m$. The storage capacity m can be finite or unbounded.

The initial state consists of the values of $V^d := [V_1, \dots, V_d]$ (for some $d < m$, usually d is 1, 2, or 3), which are set according to some probability distribution P_{V^d} (e.g. the uniform distribution).

The black-box $\mathbf{B}$ allows two types of operations, computation operations on internal state variables and queries about the internal state. No other interaction with $\mathbf{B}$ is possible. We give a more formal description of these operations:

- **Computation operations.** For a set Π of operations on S of some arities (nullary, unary, binary, or higher arity), a computation operations consist of selecting an operation $f \in \Pi$ (say t -ary) as well as the indices $i_1, \dots, i_{t+1} \leq m$ of $t+1$ state variables. $\mathbf{B}$ computes $f(V_{i_1}, \dots, V_{i_t})$ and stores the result in $V_{i_{t+1}}$.
- **Queries.** For a set Σ of relations (of some arities) on S , a query consist of selecting a relation $\rho \in \Sigma$ (say t -ary) as well as the indices $i_1, \dots, i_t \leq m$ of t state variables. The query is replied by $\rho(V_{i_1}, \dots, V_{i_t})$.

A black-box $\mathbf{B}$ is thus characterized by S , Π , Σ , m , and d . As mentioned above, one can include an abstract total order relation $\preceq$.

Example Extraction / Computation

- Extract the initial value x of V_i
- Compute $f: S^d \rightarrow S$ of the initial state within $\mathbf{B}$,
i.e. $V_i = f(x_1, \dots, x_d)$ for some i & $x_1, \dots, x_d$ are the initial values of $V_1, \dots, V_d$.

Algebraic Group Model

Definition ↓

Algebraic Algorithms

An algorithm A_{alg} executed in an algebraic game G_G is called algebraic if for all group elements Z that A_{alg} outputs, it additionally provides the representation of Z w.r.t all previously received group elements.

Let $L = L_1, \dots, L_m \in G^m$ be the list of all group elements A_{alg} has received then A_{alg} outputs $\vec{z}$ together with Z such that:

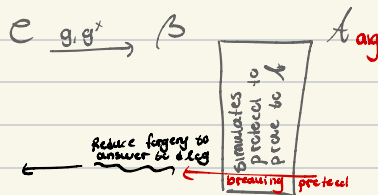
$$Z = \prod_{i=1}^m L_i^{z_i}$$

Remark about AGM in RO model:

group elements output by the RO are included in the list L .

Algebraic Group Model

- a computational model in which all adversaries are modeled as algebraic.



Comparing AGM & GGM

- Any generic algorithm can be modeled within AGM.

Let G & H be two computational problems
 A_{alg} - an algebraic algorithm that solves G

If we can convert A_{alg} via a generic reduction R_{gen} into an algorithm B_{alg} that solves H , then B_{alg} is also algebraic.

- Pairings in GGM

We also remark that proofs in the AGM have an inherently different interpretation than proofs in the GGM. To analyze the hardness of an assumption in the GGM, one must explicitly augment the model by any functionality that is offered by the structure of the group. As a simple example, let us consider a group $\mathbb{G}$ which is equipped with a symmetric bilinear map $e: \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$. The bilinear map can be modeled in the GGM via an oracle. However, it is not clear whether e can be used to gather even further information about the elements of $\mathbb{G}$. Though it is widely believed that this is not the case, a proof in the GGM provides no answer to this question, because the GGM itself is based on the conjecture that e does not offer any functionality beyond a bilinear map. In contrast, the AGM captures any such exploit without the need of having to model it explicitly and considers the relation between two problems instead of their individual hardness. This means that if one can reduce H to G in the AGM and H is conjectured to remain hard with respect to algebraic algorithms, even when given e , then also G remains hard. No similar statement can be inferred in the GGM. Thus, the AGM allows for a more fine grained assessment of the hardness of computational problems than the GGM.

Example BLS

In the AGM we can prove tight reduction of BLS in the random oracle model

Let G be a group s.t $\exists e: G \times G \rightarrow G$
 Set $\mathcal{G} = (p, G, G_T, g, e)$

BLS Signatures

<u>KeyGen(λ)</u>	<u>Sign(m, sk)</u>	<u>Verify(m, σ)</u>
$sk \xleftarrow{\$} \mathbb{Z}_p$	$\sigma = H(m)^{sk}$	check: $e(H(m), pk) \stackrel{?}{=} e(\sigma, g)$
$pk := g^x$	Return σ	
Return (sk, pk)		

<u>uf-cma_{sig, par}^A</u>	<u>$\mathcal{O}(m)$</u>
$(pk, sk) \leftarrow \text{KeyGen}$	$Q := Q \cup \{m\}$
$Q := \emptyset$	$\sigma \leftarrow \text{Sign}(m, sk)$
$(m^*, \sigma^*) \leftarrow A^{\mathcal{O}(\cdot)}(pk)$	Return σ
Return $(m^* \notin Q \wedge \text{Verify}(m^*, \sigma^*))$	

Theorem

$\text{dlog}_g \stackrel{(4,1)}{\equiv} \text{uf-cma}_{\text{BLS}, G}$ in the RO model

Remark: In standard model one would not know $\tilde{a}$...

Proof

Let A_{alg} be an algebraic adversary playing in $\text{ut-cma}_{\text{BLS}, G}^{A_{\text{alg}}}$
 $\Rightarrow$ at end A_{alg} returns $[\sigma^*]_E$ - signature of msg $m^* \notin \mathcal{Q}$.
 σ^* + representation $\tilde{a} = (\hat{a}, a', a_1, \dots, q_p, \tilde{a}_1, \dots, \tilde{a}_\ell)$

$$\sigma^* = H(m^*)^{sk} = g^{\hat{a}} pk^{a'} \prod_{i=1}^{\ell} H_i^{a_i} \prod_{i=1}^{\ell} \sigma_i^{\tilde{a}_i} \quad (*)$$

$\uparrow$ Hash queries $\text{let } H(m^*) = g^{H(m^*) \cdot g^{-1}}$
 $\uparrow$ signatures reference by sign Oracle

(*) is equivalent to

$$\begin{aligned} r^* \cdot x &= r \hat{a} + x a' + \sum r_i a_i + (\sum r_i \tilde{a}_i) \cdot x \\ &= r x (a' + \sum r_i \tilde{a}_i) + (\tilde{a} + \sum r_i a_i) \end{aligned}$$

doesn't contain $r^* \tilde{a}^*$!

$$\hookrightarrow x = (\tilde{a} + \sum r_i a_i) / \underbrace{(r - (a' + \sum r_i \tilde{a}_i))}_{\text{Extraction easy if } \neq_p 0}$$

Let B_{alg} be an algebraic adversary that with prob $\frac{1}{2}$ acts as C_{alg} & with probability $\frac{1}{2}$ acts as D_{alg} .

$C_{\text{alg}}(Z = g^z)$	$Q(m_i)$	$H(m_i)$
00 $X := \tilde{Z}$	04 If $H(m_i) = \perp$	09 If $H(m_i) \neq \perp$
01 $(m^*, [\Sigma^*]_{\tilde{a}}) \xleftarrow{\$} A^{O(\cdot), H(\cdot)}(X)$	05 $r_i \xleftarrow{\$} \mathbb{Z}_p$	10 Return $H(m_i)$
02 Compute z (see description)	06 $H(m_i) \leftarrow g^{r_i}$	11 $r_i \xleftarrow{\$} \mathbb{Z}_p$
03 Return z	07 $\Sigma_i \leftarrow X^{r_i}$	12 $H(m_i) \leftarrow g^{r_i}$
	08 Return Σ_i	13 Return $H(m_i)$

C_{alg} & D_{alg} is constructed such that C_{alg} can extract if $\star' \neq 0$ & D_{alg} if $\star = 0$.

$C_{\text{alg}}/D_{\text{alg}}$ includes challenge Z at different places

$D_{\text{alg}}(Z = g^z)$	$Q(m_i)$	$H(m_i)$
00 $x \xleftarrow{\$} \mathbb{Z}_p$	05 $\Sigma_i := H(m_i)^x$	07 $b_i, \tilde{r}_i \xleftarrow{\$} \mathbb{Z}_p$
01 $X := g^x$	06 Return Σ_i	08 Return $\underbrace{g^{z b_i + \tilde{r}_i}}_{\tilde{Z}^{b_i} \cdot \tilde{r}_i}$
02 $(m^*, [\Sigma^*]_{\tilde{a}}) \xleftarrow{\$} A^{O(\cdot), H(\cdot)}(X)$		
03 Compute z (see above)		
04 Return z		